

0.1 Anser Simulation in Python

We simulate the Anser system in python, developing on a MATLAB simulation by (WHO MADE IT?). Here's the pipeline:

0.1.1 Coil geometry

First we generate the vertex coordinates for a square spiral PCB emitter coil. This is done in a local coordinate system and then copies are rotated and translated to place them in the global coordinate system.

Each coil is a square which spirals in, it has N turns per layer, the side-length of the outermost square is l , the trace width is w , there is a line to line spacing of s and the thickness of the PCB is z_{thick} . After the coil has done N turns on the upper layer, it is connected with a via to the lower layer where the coil spirals out again.

0.1.2 Conversion from Local to Global Coordinates

As all of the coils are the same shape, only one coil is generated, then it is copied a number of times into different places in space. This is done inside `local_to_global`

The module selects an axis about which the rotation takes place. This is represented as an integer $(0, 1, 2) - (x, y, z)$. This works nicely because the rotation matrix can be generated using an even permutation of the axes. The module then applies the permuted rotation matrix to a copy of the input vector and then applies a translation.

0.1.3 Field Calculation

The calculation of the magnetic field generated by the coils is done in `local_to_global`. It uses the linearity of maxwell's equations in free space to calculate the total magnetic field due to the coils at an observer point as the sum of each straight-line segment. It then implements equation ?? to calculate the magnetic field.

0.1.4 Objective Function

The objective function takes in the real observed flux, and returns the difference between it and the flux calculated using the simulation. It calculates the simulated flux as:

$$\Phi = H_{sim} \cdot \hat{n}_{obs} \quad (1)$$

where:

- H_{sim} is the simulated magnetic field
- $\hat{n}_{obs}$ is the unit vector normal to the observer.